

Key Takeaways

1 Small Filters + Depth Beats Large Filters

Stacking 3×3 convolutions gives the same receptive field as larger filters (two $3 \times 3 =$ one 5×5 , three $3 \times 3 =$ one 7×7) with 45% fewer parameters and more non-linearity. VGG proved this principle; it underpins all modern CNN design.

2 1×1 Convolutions Are Channel Bottlenecks

A 1×1 conv mixes channels at each pixel without touching spatial dimensions. Used before expensive operations to reduce channel count ($256 \rightarrow 64$), saving up to 90% of computation. Central to both Inception modules and ResNet bottleneck blocks.

3 Skip Connections Solve the Depth Problem

Plain networks degrade when made deeper (optimization failure, not overfitting). ResNet's shortcut $y = F(x) + x$ provides a gradient highway and makes learning identity trivial. This enabled 152+ layers and superhuman ImageNet accuracy (3.6% top-5 error).

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1.** Why do three stacked 3×3 convolutions have fewer parameters than a single 7×7 , despite seeing the same receptive field?
- Q2.** A 56-layer plain network has *higher training error* than a 20-layer network. Why is this NOT overfitting? What is it?
- Q3.** What is the role of 1×1 convolutions in the Inception module? Why can't we just concatenate 3×3 and 5×5 outputs directly?
- Q4.** In a residual block, if the optimal function is close to identity, why is learning $F(x) = 0$ easier than learning $H(x) = x$ directly?
- Q5.** GoogLeNet has 5M parameters while VGG-16 has 138M, yet GoogLeNet has better accuracy. Name two architectural choices that explain this efficiency gap.

Work through these in a Colab notebook. No starter code — figure out the approach from what you learned in the lecture.

Exercise 1: Receptive Field Calculator

Write a function `receptive_field(layers)` that takes a list of (kernel_size, stride, padding) tuples and computes the total receptive field. Verify: three layers of (3, 1, 1) gives $RF = 7$. Then compute the receptive field of VGG-16's first block (two 3×3 convs + one 2×2 max pool with stride 2). Print the RF at each layer.

Exercise 2: Bottleneck Savings Calculator

Write a function that computes FLOPs for: (a) a direct convolution (in_channels $\rightarrow$ out_channels, kernel k , spatial $H \times W$), and (b) a bottleneck path (1×1 reduce $\rightarrow k \times k$ conv $\rightarrow 1 \times 1$ expand). Compare FLOPs for a 3×3 conv on $256 \rightarrow 256$ channels at 56×56 spatial vs. a bottleneck ($256 \rightarrow 64 \rightarrow 64 \rightarrow 256$). Plot savings for different reduction ratios ($2 \times$, $4 \times$, $8 \times$).

Exercise 3: Plain Net vs ResNet Depth Experiment

Build two networks: a plain CNN (stack of 3×3 conv + BN + ReLU blocks) and a ResNet (same but with skip connections). Train both on CIFAR-10 at depths 20, 32, 44, and 56 layers (10 epochs each). Plot training accuracy vs. depth for both. Does the plain network show the degradation problem? Does ResNet avoid it?

Challenge Problem

Build and Compare Mini-Architectures on CIFAR-10

Implement simplified versions of all five architectures from the lecture and compare them fairly:

1. **Mini-LeNet:** 2 conv layers (5×5), 2 FC layers, tanh activation
2. **Mini-AlexNet:** 3 conv layers (varying sizes), dropout, ReLU
3. **Mini-VGG:** 6 conv layers all 3×3 , "halve spatial, double channels"
4. **Mini-Inception:** 3 inception modules ($1 \times 1 + 3 \times 3 + 5 \times 5$ paths)
5. **Mini-ResNet:** 6 residual blocks with skip connections

Expected output: A table comparing: parameter count, test accuracy after 20 epochs, training time per epoch. Which gives the best accuracy-per-parameter ratio?

Bonus: Add global average pooling to models 3–5 and remove FC layers. Does it improve the parameter-efficiency gap?

- **LeNet-5** — LeCun et al., 1998, “*Gradient-Based Learning Applied to Document Recognition*.” The paper that started CNNs. Section 2 describes the architecture clearly.
- **AlexNet** — Krizhevsky et al., 2012, “*ImageNet Classification with Deep Convolutional Neural Networks*.” The paper that launched the deep learning era. Short and very readable.
- **VGGNet** — Simonyan & Zisserman, 2014, “*Very Deep Convolutional Networks for Large-Scale Image Recognition*.” Elegant proof that uniform 3×3 filters + depth works.
- **ResNet** — He et al., 2015, “*Deep Residual Learning for Image Recognition*.” Introduced skip connections. Read the degradation experiment in Section 4.

Online Resources & Tools

- **torchvision.models** — Pretrained implementations of all architectures covered today
- **Papers With Code: ImageNet** — Leaderboard tracking all architectures and their results
- **CS231n: CNN Architectures** — Stanford lecture notes covering LeNet through ResNet
- **CNN Explainer (Georgia Tech)** — Interactive web visualization of CNN internals
- **Netron** — Tool to visualize model architecture graphs from ONNX/PyTorch files
- **torchsummary** — Print layer-by-layer parameter counts and output shapes

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Confused about receptive fields?** Revisit Session 7 notes on how convolution kernel size, stride, and stacking determine what area of the input each neuron “sees.” The key formula: RF grows by $(k - 1)$ per layer.
- **Fuzzy on how convolution output dimensions are computed?** Formula: $\lfloor (W - K + 2P)/S \rfloor + 1$. Practice computing it for 3–4 examples until it’s automatic. Session 7 notebook has exercises for this.
- **Not sure why vanishing gradients matter here?** Revisit Session 6 (Initialization). Skip connections solve the same problem He init partially addresses — keeping gradient magnitude stable across many layers.
- **Confused about what “parameters” means for a conv layer?** Each filter has $K \times K \times C_{\text{in}}$ weights + 1 bias. A layer with C_{out} filters has $C_{\text{out}} \times (K \times K \times C_{\text{in}} + 1)$ total parameters.

Answers — Test Your Understanding

- A1.** Three 3×3 convs: $3 \times (3 \times 3 \times C^2) = 27C^2$ parameters. One 7×7 conv: $7 \times 7 \times C^2 = 49C^2$ parameters. That’s 45% fewer, plus you get 3 ReLU non-linearities instead of 1, making the function more expressive.
- A2.** Overfitting means low training error but high test error. Here **training error itself** is higher — the network can’t even fit the training data well. It’s a degradation/optimization problem: the network can’t learn the identity mapping in extra layers.
- A3.** 1×1 convs reduce channel count before expensive ops ($256 \rightarrow 64$ channels). Without them, concatenating parallel 3×3 and 5×5 outputs causes channels to explode at each module, making computation infeasible.
- A4.** Pushing all weights toward zero (giving $F(x) = 0$) is trivial with weight decay/L2. Learning $H(x) = x$ in a multi-layer block requires learning specific non-trivial weight patterns — much harder for the optimizer.
- A5.** (1) Global average pooling instead of large FC layers eliminates 100M+ parameters. (2) 1×1 bottlenecks before expensive convolutions reduce computation per inception module dramatically.